



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Cyclomatic Complexity

Cyclomatic complexity of a code section is the quantitative measure of the number of linearly independent paths in it. It is a software metric used to indicate the complexity of a program. It is computed using the Control Flow Graph of the program. The nodes in the graph indicate the smallest group of commands of a program, and a directed edge in it connects the two nodes i.e. if second command might immediately follow the first command.

For example, if source code contains no control flow statement then its cyclomatic complexity will be 1 and source code contains a single path in it. Similarly, if the source code contains one **if condition** then cyclomatic complexity will be 2 because there will be two paths one for true and the other for false.

Mathematically, for a structured program, the directed graph inside control flow is the edge joining two basic blocks of the program as control may pass from first to second.

So, cyclomatic complexity M would be defined as,

$$M = E - N + 2P$$

where,

E = the number of edges in the control flow graph

N = the number of nodes in the control flow graph

P = the number of connected components



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Steps that should be followed in calculating cyclomatic complexity and test cases design are:

- Construction of graph with nodes and edges from code.
- Identification of independent paths.
- Cyclomatic Complexity Calculation
- Design of Test Cases

Let a section of code as such:

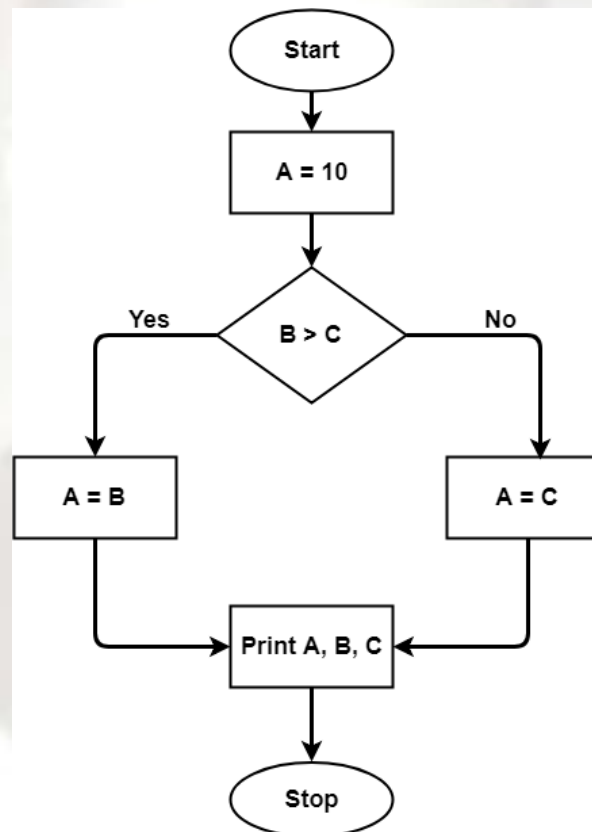
```
A = 10
IF B > C THEN
  A = B
ELSE
  A = C
ENDIF
Print A
Print B
Print C
```



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Control Flow Graph of above code





Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

The cyclomatic complexity calculated for above code will be from control flow graph. The graph shows seven shapes (nodes), seven lines (edges), hence cyclomatic complexity is $7-7+2 = 2$.

Use of Cyclomatic Complexity:

- Determining the independent path executions thus proven to be very helpful for Developers and Testers.
- It can make sure that every path have been tested at least once.
- Thus help to focus more on uncovered paths.
- Code coverage can be improved.
- Risk associated with program can be evaluated.
- These metrics being used earlier in the program helps in reducing the risks.

Advantages of Cyclomatic Complexity:

- It can be used as a quality metric, gives relative complexity of various designs.
- It is able to compute faster than the Halstead's metrics.
- It is used to measure the minimum effort and best areas of concentration for testing.
- It is able to guide the testing process.
- It is easy to apply.

Disadvantages of Cyclomatic Complexity:

- It is the measure of the programs' control complexity and not the data the data complexity.
- In this, nested conditional structures are harder to understand than non-nested structures.
- In case of simple comparisons and decision structures, it may give a misleading figure.



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Cyclomatic Complexity may be defined as-

- It is a software metric that measures the logical complexity of the program code.
- It counts the number of decisions in the given program code.
- It measures the number of linearly independent paths through the program code.

Cyclomatic complexity indicates several information about the program code-

Cyclomatic Complexity	Meaning
1 – 10	• Structured and Well Written Code • High Testability • Less Cost and Effort
10 – 20	• Complex Code • Medium Testability • Medium Cost and Effort
20 – 40	• Very Complex Code • Low Testability • High Cost and Effort



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

> 40

- Highly Complex Code
- Not at all Testable
- Very High Cost and Effort

Importance of Cyclomatic Complexity-

- It helps in determining the software quality.
- It is an important indicator of program code's readability, maintainability and portability.
- It helps the developers and testers to determine independent path executions.
- It helps to focus more on the uncovered paths.
- It evaluates the risk associated with the application or program.
- It provides assurance to the developers that all the paths have been tested at least once.

Properties of Cyclomatic Complexity-

- ✚ It is the maximum number of independent paths through the program code.
- ✚ It depends only on the number of decisions in the program code.
- ✚ Insertion or deletion of functional statements from the code does not affect its cyclomatic complexity.
- ✚ It is always greater than or equal to 1.



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Calculating Cyclomatic Complexity-

Cyclomatic complexity is calculated using the control flow representation of the program code.

In control flow representation of the program code,

Nodes represent parts of the code having no branches.

Edges represent possible control flow transfers during program execution

There are 3 commonly used methods for calculating the cyclomatic complexity-

Method-01:

Cyclomatic Complexity = Total number of closed regions in the control flow graph + 1

Method-02:

Cyclomatic Complexity = $E - N + 2$

Here-

E = Total number of edges in the control flow graph

N = Total number of nodes in the control flow graph



Software Engineering vs Computer Science

SE-II13 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Method-03:

Cyclomatic Complexity = $P + 1$

Here,

P = Total number of predicate nodes contained in the control flow graph

Note-

Predicate nodes are the conditional nodes.

They give rise to two branches in the control flow graph.

PRACTICE PROBLEMS BASED ON CYCLOMATIC COMPLEXITY-

Problem-01:

Calculate cyclomatic complexity for the given code-

```
IF A = 354
  THEN IF B > C
    THEN A = B
    ELSE A = C
  END IF
END IF
PRINT A
```

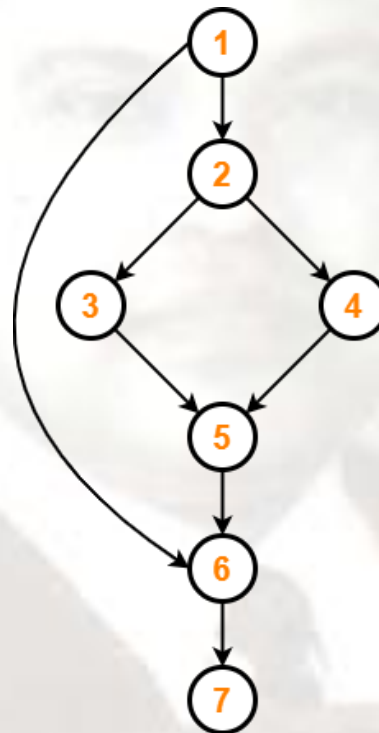


Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Solution-

We draw the following control flow graph for the given code-



Control Flow Graph



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Using the above control flow graph, the cyclomatic complexity may be calculated as-

Method-01:

Cyclomatic Complexity

$$\begin{aligned} &= \text{Total number of closed regions in the control flow graph} + 1 \\ &= 2 + 1 \\ &= 3 \end{aligned}$$

Method-02:

Cyclomatic Complexity

$$\begin{aligned} &= E - N + 2 \\ &= 8 - 7 + 2 \\ &= 3 \end{aligned}$$

Method-03:

Cyclomatic Complexity

$$\begin{aligned} &= P + 1 \\ &= 2 + 1 \\ &= 3 \end{aligned}$$



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Problem-02:
Calculate cyclomatic complexity for the given code-

```
{ int i, j, k;  
  for (i=0 ; i<=N ; i++)  
    p[i] = 1;  
  for (i=2 ; i<=N ; i++)  
  {  
    k = p[i]; j=1;  
    while (a[p[j-1]] > a[k] {  
      p[j] = p[j-1];  
      j--;  
    }  
    p[j]=k;  
  }  
}
```

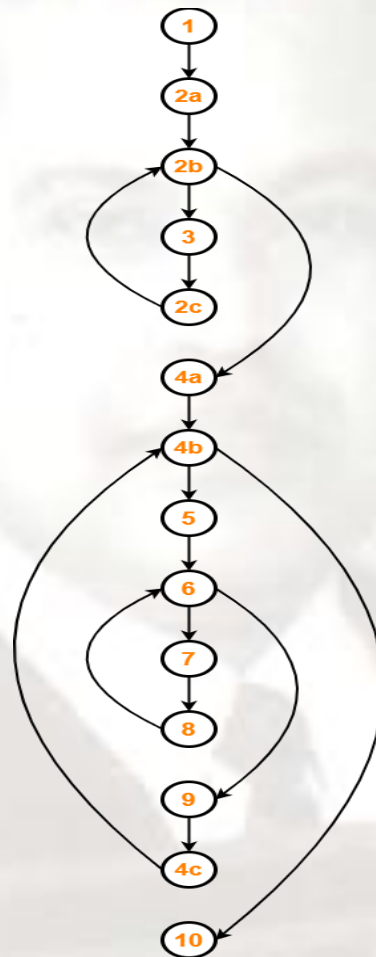
Solution-

We draw the following control flow graph for the given code-



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)



Control Flow Graph



Software Engineering vs Computer Science

SE-II13 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Using the above control flow graph, the cyclomatic complexity may be calculated as-

Method-01:
Cyclomatic Complexity

$$\begin{aligned} &= \text{Total number of closed regions in the control flow graph} + 1 \\ &= 3 + 1 \\ &= 4 \end{aligned}$$

Method-02:
Cyclomatic Complexity

$$\begin{aligned} &= E - N + 2 \\ &= 16 - 14 + 2 \\ &= 4 \end{aligned}$$

Method-03:
Cyclomatic Complexity

$$\begin{aligned} &= P + 1 \\ &= 3 + 1 \\ &= 4 \end{aligned}$$



Software Engineering vs Computer Science

SE-II13 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Problem-03:
Calculate cyclomatic complexity for the given code-

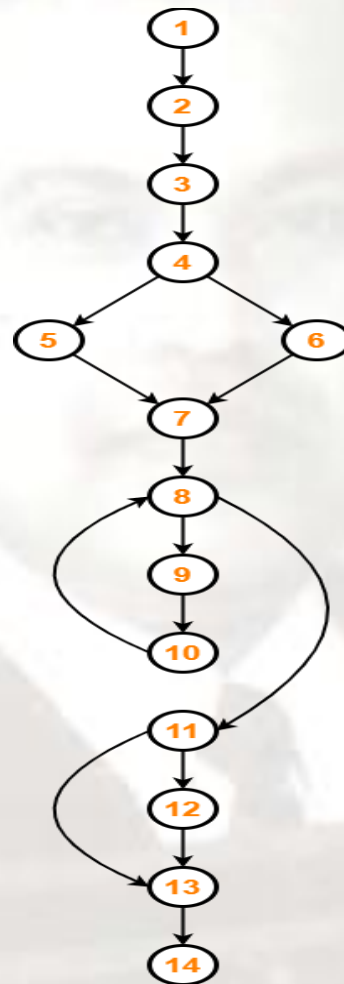
```
begin int x, y, power;  
  float z;  
  input(x, y);  
  if(y<0)  
    power = -y;  
  else power = y;  
  z=1;  
  while(power!=0)  
  {  z=z*x;  
      power=power-1;  
  } if(y<0)  
    z=1/z;  
  output(z);  
end
```

Solution-
We draw the following control flow graph for the given code-



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)





Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

Using the above control flow graph, the cyclomatic complexity may be calculated as-

Method-01:

Cyclomatic Complexity

= Total number of closed regions in the control flow graph + 1

= 3 + 1

= 4

Method-02:

Cyclomatic Complexity

= $E - N + 2$

= $16 - 14 + 2$

= 4

Method-03:

Cyclomatic Complexity

= $P + 1$

= 3 + 1

= 4



Software Engineering vs Computer Science

SE-113 Introduction to Software Engineering Course Teacher: S A M Matiur Rahman (SMR)

QA perspective:

To understand the importance of Cyclomatic complexity from a QA perspective, the result we get from the formula determines how much testing is required to test that piece of source code. For example, if the number is higher, that piece of code requires in-depth testing compared to the code which has lower Cyclomatic complexity. Our testing scope and the methods would be defined according to the complexity of the functionality. The control flow graph for the code under test can be obtained from the developer to calculate the number of edges, nodes and exit points. Also, a number of test cases for maximum test coverage can be determined as well based on the paths in the CFG (control flow graph) and the branches. In other words, Cyclomatic complexity is very helpful for QA to estimate the scope of testing, determine how skilled tester is required, and the extent of testing required for the best way possible code coverage. Also, it is one of the best ways with which developers can make QA team to understand the source code complexity.

Conclusion:

Cyclomatic complexity calculation is one of the standard approaches to calculate the source code complexity and determine the risk that the source code possesses for any future modification and maintenance. Lower the Cyclomatic complexity better is the code quality in terms of complexity.

Reference:

1. https://en.wikipedia.org/wiki/Cyclomatic_complexity
2. [Cyclomatic Complexity with Example - Software Testing Class](#)
3. <https://www.gatevidyalay.com/cyclomatic-complexity-calculation-examples/>